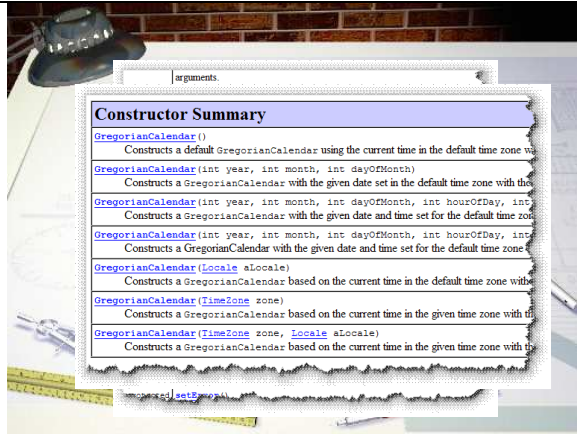




Slide 1

CS 170

Java Programming 1

Overloading

Duration: 00:01:07

Advance mode: Auto

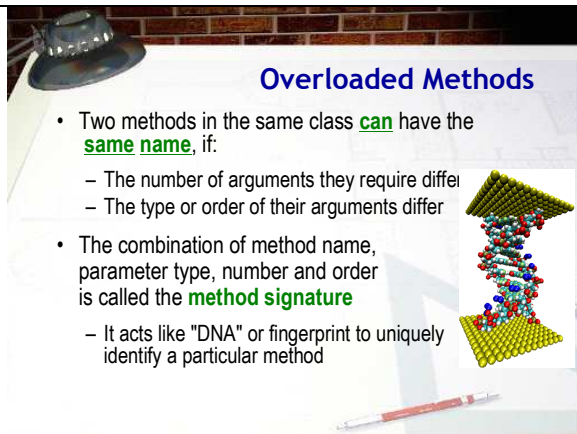
Notes:

Hi there. Welcome to the CS 170, Java Programming 1 lecture on method overloading.

In this class, the first object that you met was the global `PrintStream` object, `System.out`, and the first method that you used was its `println()` method. If you look up the documentation for `PrintStream` class, though, you'll find not one, but ten different versions of `println()`, each one customized to display a different kind of output.

A similar situation occurred when you worked with the `GregorianCalendar` class in Chapter 2. When it came time to create a `GregorianCalendar`, you found you had more than a half-dozen different versions of the constructor you could use, initializing the date to the system date and time, a specific date or time or even a particular country and time zone.

These kinds of constructors and methods, that use the same name but process different kinds of data, are called overloaded methods, and that's the topic of this lecture.



Slide 2

Overloaded Methods

Duration: 00:00:58

Advance mode: Auto

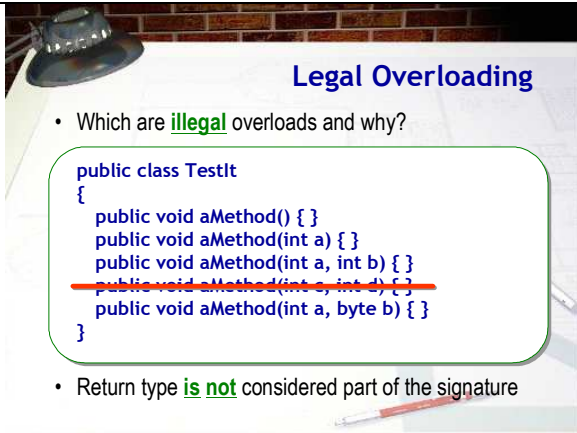
Notes:

You might be surprised to find out that the same class can have two methods that have the same name. In fact, you can have several methods, in the same class, all sharing the same name; it's quite common. Such methods are called overloaded methods.

Here is the basic rule that determines how such methods can be created and which particular version of a method will be called. Methods in the same class may have the same name, if:

- The number of their formal parameters differ
- The type or order of their formal parameter types differ

This combination of method name, formal parameter type, number, and order, is called the method's signature, and it acts like "DNA" to uniquely identify a particular method.



Legal Overloading

- Which are **illegal** overloads and why?

```
public class TestIt
{
    public void aMethod() {}
    public void aMethod(int a) {}
    public void aMethod(int a, int b) {}
    public void aMethod(int a, int d) {}
    public void aMethod(int a, byte b) {}
}
```

- Return type **is not** considered part of the signature

Slide 3

Legal Overloading

Duration: 00:01:10
Advance mode: Auto

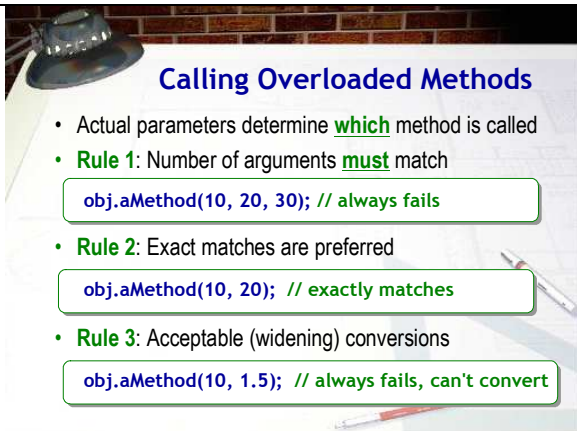
Notes:

Here is a class with several methods, all having the same name. Take a look at the different method signatures, and, using the rules you've just learned, decide which methods are legal, and which methods are illegal (assuming that all of them appeared in the same class, just as shown here).

Of the five methods here, numbers 1, 2, and 5 are legal, while number 3 and 4 are not, because they have the same number and type of parameters. That means to make this class compile, we'd need to delete either method three or method four, like this.

Note that when we look at the documentation for a method, it always includes the return type. For purposes of method overloading, though, the return type just doesn't count. That is, you can't have two methods with the same name, the same number and type of arguments, but a different return type. That's just not permitted.

So, how does Java decide which method to use? Through a process called overloading resolution. Let's look at that now.



Calling Overloaded Methods

- Actual parameters determine **which** method is called
- Rule 1:** Number of arguments **must** match


```
obj.aMethod(10, 20, 30); // always fails
```
- Rule 2:** Exact matches are preferred


```
obj.aMethod(10, 20); // exactly matches
```
- Rule 3:** Acceptable (widening) conversions


```
obj.aMethod(10, 1.5); // always fails, can't convert
```

Slide 4

Calling Overloaded Methods

Duration: 00:02:39
Advance mode: Auto

Notes:

When calling a method, Java looks at the actual arguments to decide which of several overloaded methods to call. The rules for matching actual argument with the formal parameters can become quite baroque, but here's the simple version. For this example, let's assume we have a version of the TestIt class, as shown on the previous slide, with the fourth method removed. Let's assume as well, that you have created a TestIt object named obj by using the TestIt constructor. Here are the rules that will apply when you try to call any of the methods in the TestIt class using your object obj.

Rule #1 is that the number of arguments or parameters must match. When you call a method, Java first tries to match the number of actual arguments with a method that has the same number of formal parameters. If it cannot find a match, then it gives up as an error. Thus the method call: obj.aMethod(10, 20, 30); shown here immediately fails, because the TestIt class has no methods that take three arguments.

Rule #2 is that exact type matches are preferred. Once Java has located a candidate method, (or methods), having the correct number of formal parameters, it compares the type of each actual argument to the corresponding formal parameter. If an exact match can be found, then Java calls the method. Thus the method call: obj.aMethod(10, 20); exactly matches the third method, because both arguments are of type int.

Rule #3 is that acceptable widening conversions will be used if no exact match can be found. If Java cannot match the exact types of arguments, it looks for acceptable conversions, preferring the conversions that are closest to the actual argument type. Java will not, however, make conversions where data will be lost. Thus the call: obj.aMethod(10, 1.5);

will always fail because 1.5 cannot be converted to either an int or a byte, the only two options for the second argument, given the available methods. However, calling `obj.aMethod(a, b)` where `a` and `b` were both bytes would be acceptable, since the first byte parameter could be converted to an int to get a match.



Good Arguments

- Here are a list of acceptable (widening) conversions

Formal Parameters	Acceptable	Unacceptable
byte	byte	everything else
short	byte, short	everything else
int	byte, short, int, char	everything else
long	byte, short, int, char, long	float, double
char	byte, short	everything else
long	byte, short, int, char, long	float, double
float	float, all integer types	double
double	all numeric types	none

Slide 5
Good Arguments
Duration: 00:01:39
Advance mode: Auto

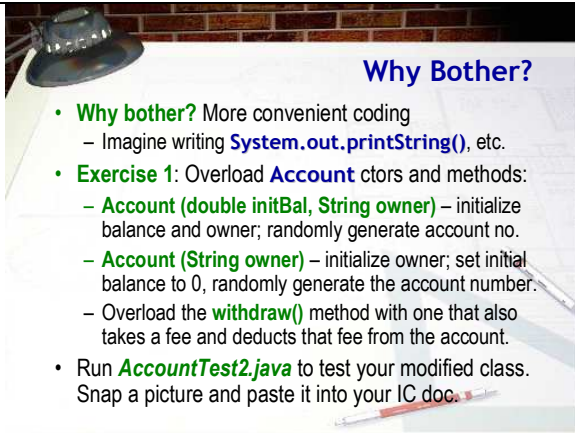
Notes:

When an exact match for your actual arguments can't be found, then the actual arguments you pass must be at least assignment compatible with the formal parameters. Java will attempt to ensure this using a process called type-checking.

Generally what this means is that if assignment would succeed then the method call will also succeed. Remember, you can normally assign a smaller or less precise value to a more precise variable.

Here are some examples of acceptable primitive numeric arguments that may help you:

- If a formal method parameter is a byte, then there are no acceptable conversions. Only a byte variable will do.
- If a formal parameter is a short, then either byte or short are acceptable. You can't use int or anything else.
- If a formal parameter is int, then you can use int, short and byte as well as char.
- If the formal parameter is long, then you can use any of the integer types as well as char. You can't use float or double, though.
- If the type parameter is char (which is really a 16-bit number), you can also pass byte and short.
- If the type is float, then you can use any integer type, but you can't use double.
- Finally, if the formal parameter is of type double, then you can pass any numeric type and it will be converted successfully.



Why Bother?

- **Why bother?** More convenient coding
 - Imagine writing `System.out.println()`, etc.
- **Exercise 1:** Overload `Account` ctors and methods:
 - `Account (double initBal, String owner)` – initialize balance and owner; randomly generate account no.
 - `Account (String owner)` – initialize owner; set initial balance to 0, randomly generate the account number.
 - Overload the `withdraw()` method with one that also takes a fee and deducts that fee from the account.
- Run `AccountTest2.java` to test your modified class. Snap a picture and paste it into your IC doc.

Slide 6

Why Bother?

Duration: 00:02:17

Advance mode: Auto

Notes:

Why bother with method overloading? Doesn't it seem like having multiple methods, all with the same name, would be confusing? How are you supposed to remember which method is being called? All of these are valid arguments against using overloaded methods. Simply giving all of your methods the same name without ample planning and forethought really will make your programs harder to understand and maintain.

Properly used, however, method overloading allows you to write methods that are much easier to use than those that don't employ overloading. Imagine, for instance, if there was no method overloading. That means when you wanted to print a String, you'd need to use the `System.out.println()` method, when you wanted to print an int, `System.out.println()` and to print a double, you'd need a `System.out.printDouble()`. Obviously, it's much more convenient, just to remember `System.out.println()` for everything you want to print.

OK, now it's your turn. Create a new section in your IC exercise document. Then, open Eclipse and the `Account` class and `AccountTest2.java`. Notice that `AccountTest2` has several errors. To get it to run, make these modifications:

- Add the overloaded constructor: `Account(double, String)`. Use the parameters to initialize balance and owner. Randomly generate an account number.
- Add a second overloaded constructor: `Account(String)`. Use the parameter to initialize owner. Set the balance to 0 and randomly generate the account number.
- Provide an overloaded version of the `withdraw()` method that subtracts a fee as well as the amount requested.

When you've done that, run `AccountTest2.java` and supply some reasonable values to test your class. Shoot me a screenshot of your code and then give me a picture of your program running.